

Assembly Utility Toolkit

A four-in-one menu driven program written in 16-bit x86 assembly

<https://github.com/hasanshahidd/assembly-utility-toolkit>

Course	Computer Organization & Assembly Language (COAL)
Language	x86 Assembly (NASM syntax), targeting 16-bit 8086
Toolchain	NASM 2.x + ALINK v1.6
Runtime	DOSBox 0.74 on Windows 11
Lines of code	Around 550 across five .asm files
Modules	File viewer, number converter, system clock, XOR cipher

Team

Member	Roll Number	Role
Sara Hanif	082	Modules: clock, number converter, encryption
Hassan Shahid	056	Modules: file viewer, main menu, build script

What this project is

This is a small command line program for DOS, written from scratch in assembly language. It pulls up a menu, you press a number, and one of four little utilities runs. There is no C, no standard library, no high level helper of any kind. Everything happens through CPU instructions and DOS interrupts.

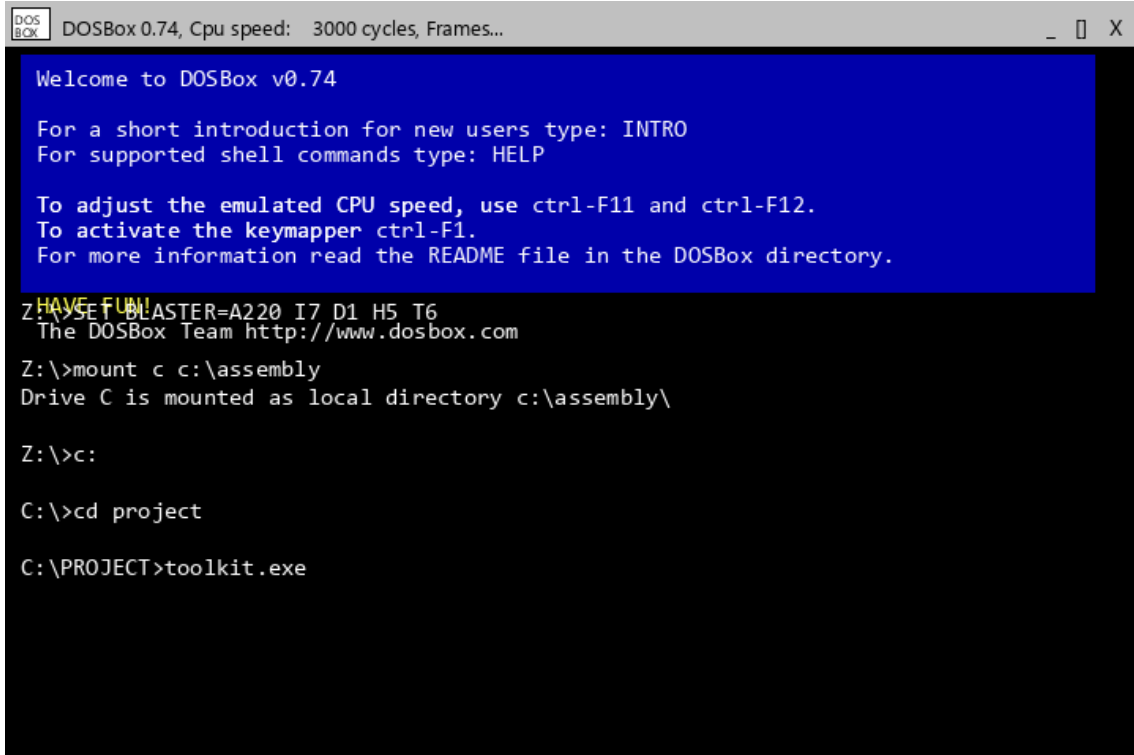
The idea was to take a handful of independent assembly programs we wrote during the semester and stitch them into one polished tool. The result is five separate source files that get assembled into object files and then linked together into a single executable called **toolkit.exe**. Each module lives in its own file with its own code and data segments, which keeps the project tidy and makes it easy to keep working on one piece without touching the others.

The four utilities at a glance

Key	Module	What it does
1	File Viewer	Asks for a filename, opens the file with int 21h/3Dh, reads 128 bytes at a time and prints the
2	Number Converter	Reads a decimal number up to 65535, then prints the same value in hexadecimal and 16-b
3	System Clock	Calls DOS int 21h/2Ch, picks up hours, minutes, and seconds out of CH, CL, and DH, print
4	XOR Encryption	Takes a short message and a numeric key, XORs each byte, prints the cipher in hex, then
5	Exit	Returns control to the DOS prompt with int 21h/4C00h.

The toolkit in action

All screenshots were captured live while the program was running inside DOSBox. Mount the project folder, switch to it, and run **toolkit.exe**. The first thing you see is the familiar blue DOSBox banner, followed by our menu.

A screenshot of a DOSBox window. The title bar says "DOSBox 0.74, Cpu speed: 3000 cycles, Frames...". The main window has a blue background with white text. It displays a welcome message, instructions for new users, and supported shell commands. Below this, it shows the command prompt at Z:\> where the user mounts drive C to c:\assembly and then runs toolkit.exe from the project directory.

```
DOS
BOX
DOSBox 0.74, Cpu speed: 3000 cycles, Frames...

Welcome to DOSBox v0.74

For a short introduction for new users type: INTRO
For supported shell commands type: HELP

To adjust the emulated CPU speed, use ctrl-F11 and ctrl-F12.
To activate the keymapper ctrl-F1.
For more information read the README file in the DOSBox directory.

Z:\>SET BLASTER=A220 I7 D1 H5 T6
The DOSBox Team http://www.dosbox.com

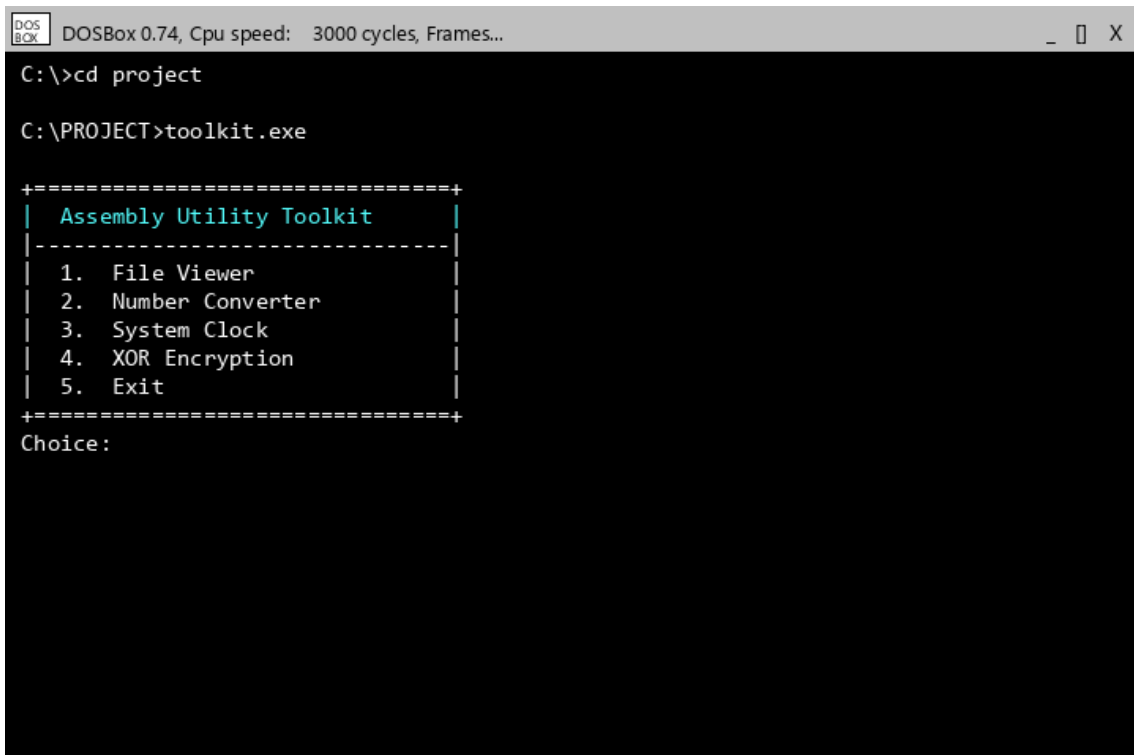
Z:\>mount c c:\assembly
Drive C is mounted as local directory c:\assembly\

Z:\>c:

C:\>cd project

C:\PROJECT>toolkit.exe
```

Starting DOSBox, mounting the C drive to our project folder, and launching toolkit.exe.

A screenshot of a DOSBox window showing the main menu of the Assembly Utility Toolkit. The title bar is the same as the previous screenshot. The command prompt shows the user has navigated to the project directory and run toolkit.exe. The program displays a menu with five options: File Viewer, Number Converter, System Clock, XOR Encryption, and Exit. The prompt "Choice:" is visible at the bottom.

```
DOS
BOX
DOSBox 0.74, Cpu speed: 3000 cycles, Frames...

C:\>cd project

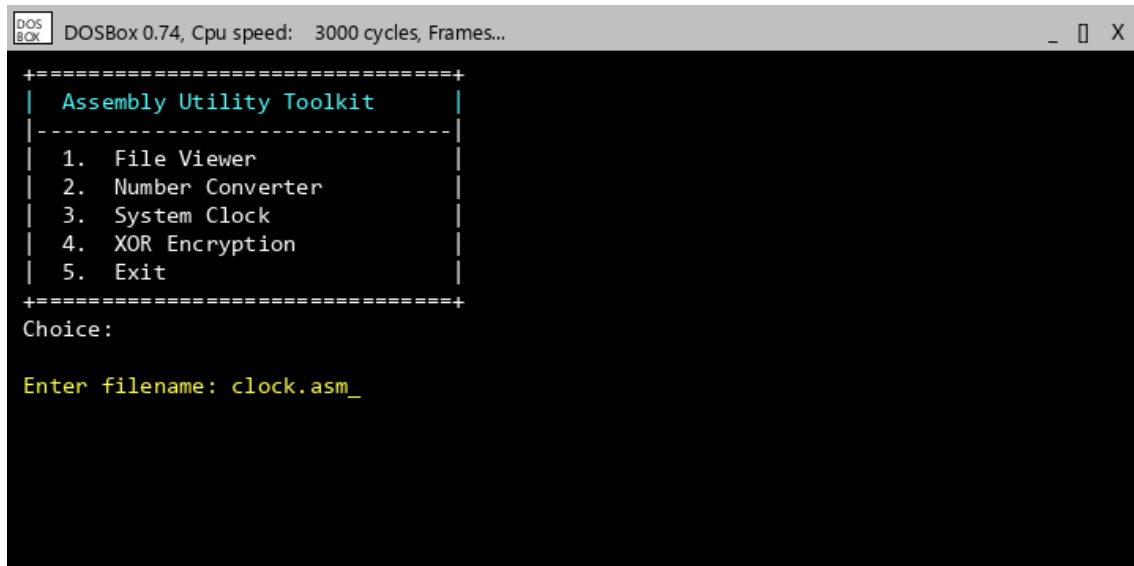
C:\PROJECT>toolkit.exe

+-----+
|  Assembly Utility Toolkit  |
+-----+
| 1. File Viewer             |
| 2. Number Converter        |
| 3. System Clock           |
| 4. XOR Encryption          |
| 5. Exit                   |
+-----+
Choice:
```

The main menu, drawn entirely with int 21h/09h string prints. A single keypress chooses a module.

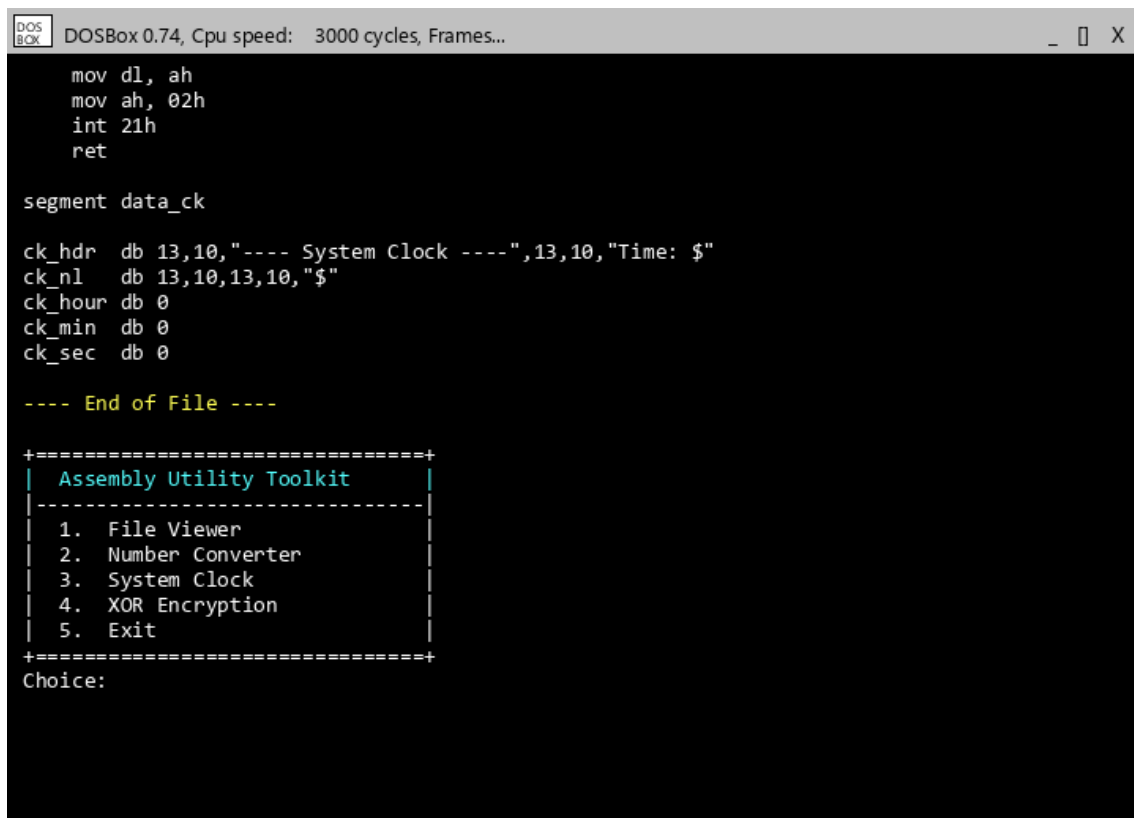
File Viewer

We tested the file viewer by asking it to show **clock.asm**, one of the other module sources. It asks for a filename, opens the file in read-only mode, and streams the contents to the screen 128 bytes at a time until it sees end of file.



```
DOS BOX DOSBox 0.74, Cpu speed: 3000 cycles, Frames...
+=====+
|  Assembly Utility Toolkit  |
+-----+
|  1. File Viewer           |
|  2. Number Converter      |
|  3. System Clock         |
|  4. XOR Encryption       |
|  5. Exit                 |
+-----+
Choice:
Enter filename: clock.asm_
```

The viewer prompts for a filename.



```
DOS BOX DOSBox 0.74, Cpu speed: 3000 cycles, Frames...
mov dl, ah
mov ah, 02h
int 21h
ret

segment data_ck

ck_hdr db 13,10,"---- System Clock ----",13,10,"Time: $"
ck_n1 db 13,10,13,10,"$"
ck_hour db 0
ck_min db 0
ck_sec db 0

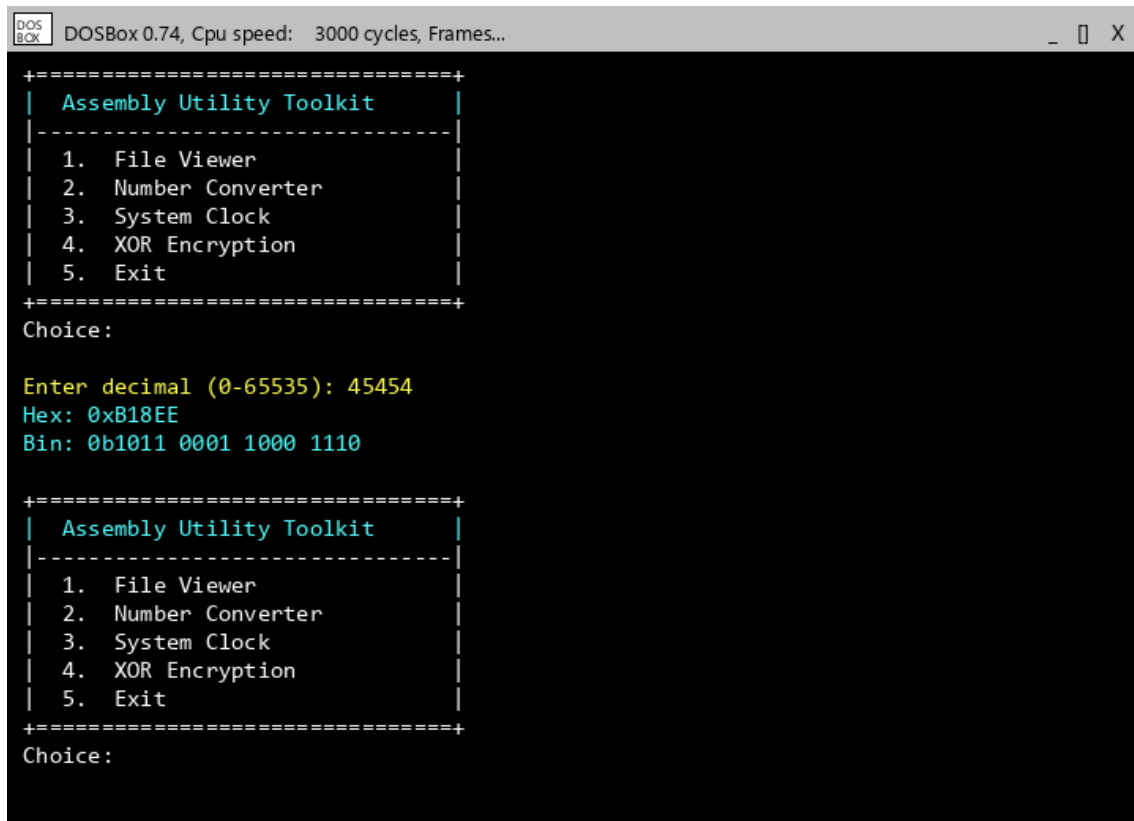
---- End of File ----

+=====+
|  Assembly Utility Toolkit  |
+-----+
|  1. File Viewer           |
|  2. Number Converter      |
|  3. System Clock         |
|  4. XOR Encryption       |
|  5. Exit                 |
+-----+
Choice:
```

Tail end of clock.asm rendered on screen, followed by an End of File marker, then back to the main menu.

Number Converter

The converter accepts any decimal value from 0 up to 65535 (the maximum a 16-bit register can hold). It builds the value by parsing one digit at a time, multiplying the running total by ten. The hex output comes from shifting right by 12, 8, 4, and 0 bits and masking the low nibble. The binary output walks the 8000h mask from bit 15 down to bit 0.



```
DOSBox 0.74, Cpu speed: 3000 cycles, Frames...
+=====+
|  Assembly Utility Toolkit  |
+-----+
|  1. File Viewer           |
|  2. Number Converter      |
|  3. System Clock         |
|  4. XOR Encryption       |
|  5. Exit                 |
+-----+
Choice:

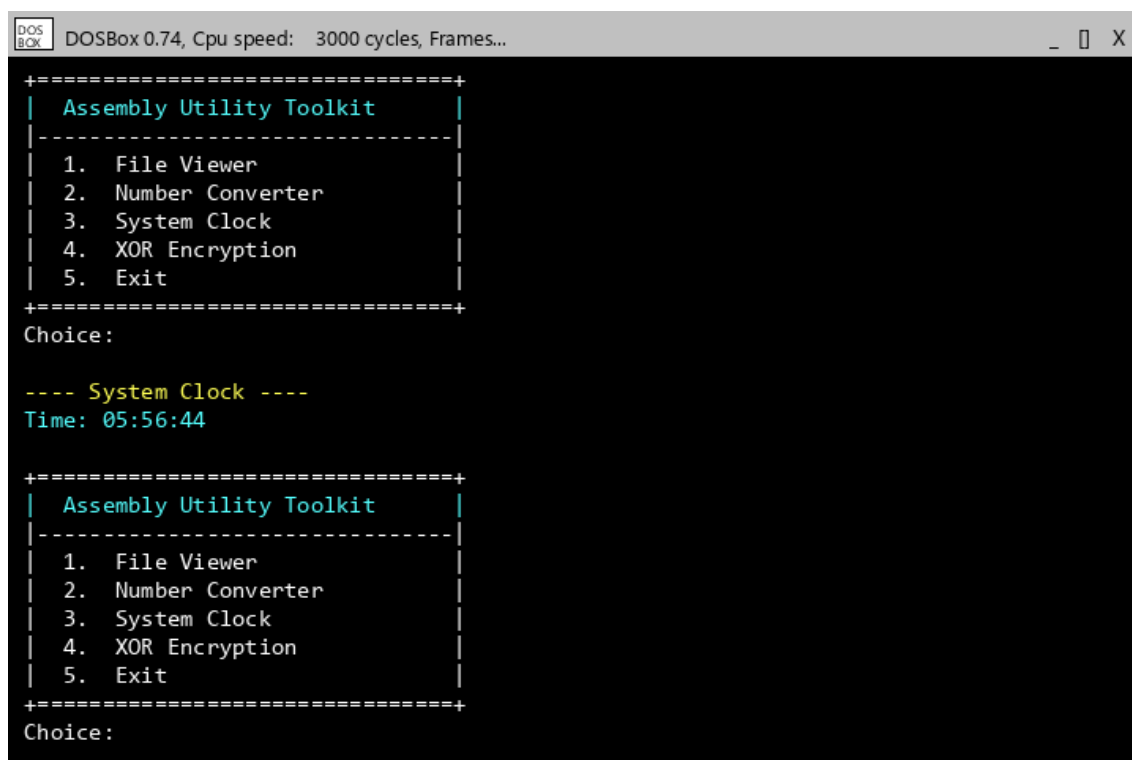
Enter decimal (0-65535): 45454
Hex: 0xB18EE
Bin: 0b1011 0001 1000 1110

+=====+
|  Assembly Utility Toolkit  |
+-----+
|  1. File Viewer           |
|  2. Number Converter      |
|  3. System Clock         |
|  4. XOR Encryption       |
|  5. Exit                 |
+-----+
Choice:
```

Entering 45454 and getting back the matching hex and binary.

System Clock

Reading the time on DOS is delightfully simple. One interrupt call returns the hour, minute, and second in separate CPU registers. The module formats each one as two decimal digits with a quick divide by ten, and prints them with colons in between.



```
DOSBox 0.74, Cpu speed: 3000 cycles, Frames...
+=====+
|  Assembly Utility Toolkit  |
+-----+
|  1. File Viewer           |
|  2. Number Converter      |
|  3. System Clock         |
|  4. XOR Encryption       |
|  5. Exit                 |
+-----+
Choice:

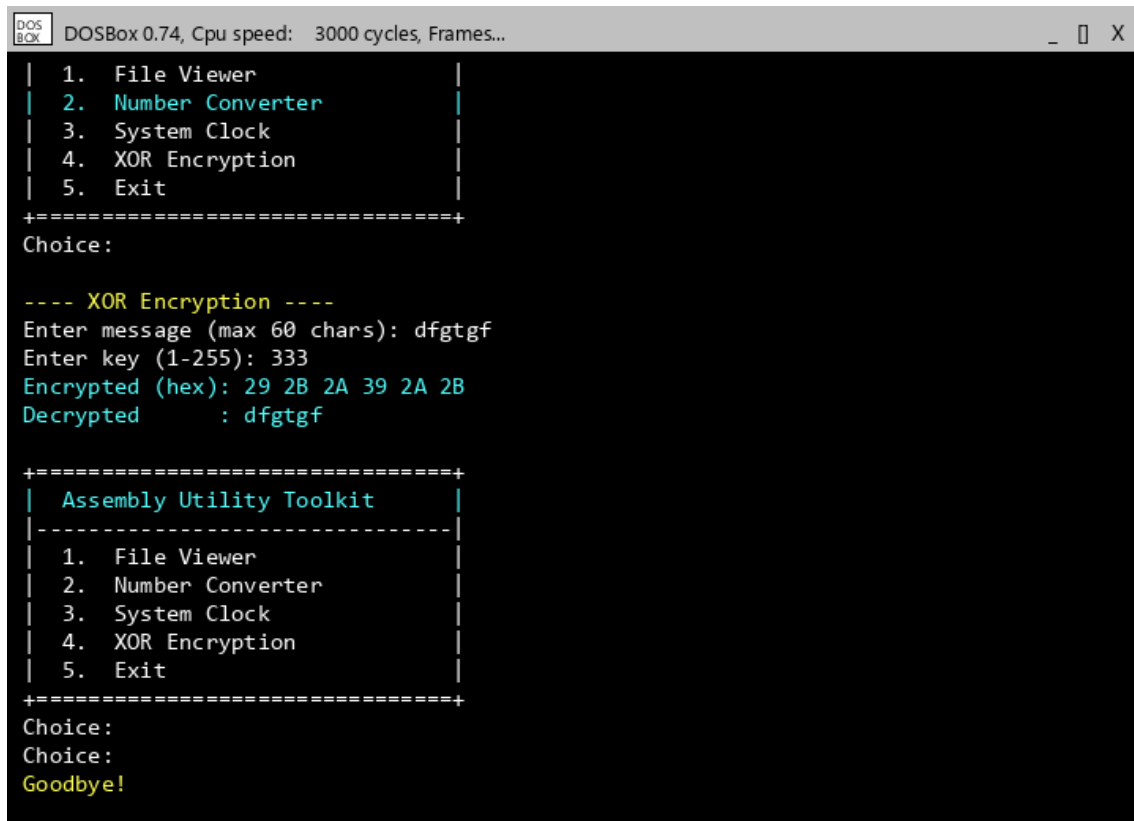
---- System Clock ----
Time: 05:56:44

+=====+
|  Assembly Utility Toolkit  |
+-----+
|  1. File Viewer           |
|  2. Number Converter      |
|  3. System Clock         |
|  4. XOR Encryption       |
|  5. Exit                 |
+-----+
Choice:
```

The current system time, printed as HH:MM:SS.

XOR Encryption

This module shows the classic property of the XOR cipher: encrypting and decrypting use the exact same operation. The user types a short message and a numeric key. Every byte of the message is XORed with the key byte, the cipher is printed as hex, and then the same XOR is applied again to recover the original text. It is not secure encryption, but it is the right size for a teaching example.



```
DOSBox 0.74, Cpu speed: 3000 cycles, Frames...
| 1. File Viewer |
| 2. Number Converter |
| 3. System Clock |
| 4. XOR Encryption |
| 5. Exit |
+=====+
Choice:

---- XOR Encryption ----
Enter message (max 60 chars): dfgtgf
Enter key (1-255): 333
Encrypted (hex): 29 2B 2A 39 2A 2B
Decrypted : dfgtgf

+=====+
| Assembly Utility Toolkit |
|-----|
| 1. File Viewer |
| 2. Number Converter |
| 3. System Clock |
| 4. XOR Encryption |
| 5. Exit |
+=====+
Choice:
Choice:
Goodbye!
```

Encrypting the message dfgtgf with key 333, then a graceful exit back to the DOS prompt.

How to build and run it

You only need three things on your machine: NASM (the assembler), ALINK (the linker), and DOSBox (so a 16-bit program can actually run on modern 64-bit Windows).

Prerequisites

- NASM, available from *nasm.us*.
- ALINK v1.6 linker. We keep it at C:\Assembly\ALINK.EXE.
- DOSBox, available from *dosbox.com*.

Building with the helper script

The repository contains a **build.bat** that handles both steps in one go. Open a Windows command prompt and run:

```
cd C:\Assembly\project
build.bat
```

On success the script prints **BUILD SUCCESS!** and leaves a fresh **toolkit.exe** beside the source files.

Building by hand

```
nasm -f obj main.asm -o main.obj
nasm -f obj fileview.asm -o fileview.obj
nasm -f obj converter.asm -o converter.obj
nasm -f obj clock.asm -o clock.obj
nasm -f obj encrypt.asm -o encrypt.obj
alink -oPE main.obj fileview.obj converter.obj clock.obj encrypt.obj toolkit.exe
```

Running it in DOSBox

```
mount c c:\Assembly
c:
cd project
toolkit.exe
```


How the code is organised

The project is split across five short assembly files. **main.asm** is the entry point and only knows about the menu and the four module functions. The four module files each export exactly one global symbol that the main loop calls. ALINK is responsible for stitching the five object files together and resolving those symbols.

File	Type	Role
main.asm	.asm	Sets up segments, draws the menu, reads one key, dispatches to the chosen module.
fileview.asm	.asm	Module 1. Opens, reads, and prints a file.
converter.asm	.asm	Module 2. Decimal to hex and binary.
clock.asm	.asm	Module 3. Reads the system clock and prints it.
encrypt.asm	.asm	Module 4. XOR encryption and decryption.
build.bat	.bat	Assembles everything and links it in one command.
toolkit.exe	.exe	The final executable that you actually run.

DOS interrupts we used

Everything visible on the screen, every byte read from disk, and every system fact we know comes from a DOS interrupt. Here is the full catalogue used by the four modules.

Call	AH	What it does	Used by
int 21h	02h	Print one character (DL holds it)	fileview, clock, encrypt
int 21h	08h	Read a single key without echoing it	main menu
int 21h	09h	Print a \$-terminated string	every module
int 21h	0Ah	Buffered keyboard input (a whole line)	fileview, encrypt
int 21h	2Ch	Read the system time into CH, CL, DH	clock
int 21h	3Dh	Open a file in read-only mode	fileview
int 21h	3Eh	Close an open file handle	fileview
int 21h	3Fh	Read bytes from a file handle	fileview
int 21h	4C00h	Exit the program and return control to DOS	main menu

Assembly concepts the project exercises

Concept	Where it shows up
BITS 16 and CPU 8086	Every source file. Forces NASM to emit pure 8086 code so the binary runs on any DOS computer.
Segmented memory	Each module owns a code segment and a private data segment, plus main.asm has its own segments.
global / extern	main.asm declares extern for each module function, the modules declare global for the same functions.

push / pop	Every module preserves DS, ES, and BP before doing anything interesting, then restores the
Local labels	Names that begin with a dot belong only to the enclosing function, so the same label name c
shr / shl by CL	The 8086 cannot shift by an arbitrary immediate, so the converter loads CL with the shift cou
Carry flag (jc)	After int 21h/3Dh, the fileview checks the carry flag to detect file-open failures.
loop and div	Used throughout to walk byte buffers and to split numbers into digits for printing.

Closing thoughts

What we like about this project is how small it is. The whole thing fits in roughly 550 lines of assembly across five files, and yet it demonstrates almost every concept the course has covered: segments, the stack, system calls, file I/O, arithmetic, bit manipulation, loops, conditional jumps, and a tiny taste of cryptography.

There are no shortcuts hidden anywhere. Every character you see on the screen got there because the program loaded it into DL and invoked int 21h with AH equal to 02h. Every file byte came back from a hand-written read loop. The point of the exercise was to feel what a real program looks like when there is no operating system abstraction between your code and the machine. We think this version does that.

Repository

The complete source, build script, screenshots, and this report live at <https://github.com/hasanshahidd/assembly-utility-toolkit>. Clone it, fork it, or just browse the code.

Submitted by

Name	Roll Number
Sara Hanif	082
Hassan Shahid	056